

VITYARTHI PROJECT REPORT

CampusLab

Laboratory Resource & Experiment Booking Manager

Programming in Java | Core Java Command-Line Application

Student Name	Love Chaudhari
Registration No.	25BAI11292
Program	B.Tech CSE - AI & ML
Course	Programming in Java
Academic Session	2025-2029
Institution	VIT Bhopal University
Department	Computer Science & Engineering

1. Introduction

CampusLab is a command-line Java application designed to manage shared laboratory resources, experiments, bookings, resource issue and return, and maintenance records in a college environment. The project was developed as a semester project for Programming in Java, with the main goal of applying the language features and object-oriented programming concepts taught in the course to a practical problem.

The application focuses on a realistic situation in which laboratory equipment and experiment slots are limited and must be shared among students. Instead of maintaining paper registers or disconnected spreadsheets, CampusLab provides a structured way to check availability, submit booking requests, validate conflicts, record resource usage, and maintain basic history.

The project uses Core Java and a command-line interface so that the implementation remains centered on Java programming rather than on web frameworks or unnecessary external technologies. JDBC is used for database persistence, while file I/O supports reports and backup operations.

2. Problem Statement

College laboratories commonly have a limited number of computers, experiment kits, measuring instruments, and other shared resources. In a manual process, students may not know which resources are available, staff may have to check several records before approving a request, and overlapping bookings can occur.

The problem is therefore to create a simple and reliable application that can maintain laboratory resources and experiments, accept booking requests, detect conflicts, track resource usage, and record maintenance activities while demonstrating the Java concepts taught in the course.

3. Objectives

1. Provide a structured system for managing laboratory resources and experiments.
2. Allow students to check availability and submit booking requests.
3. Prevent invalid or conflicting bookings through validation and business rules.
4. Allow staff to manage resources, approve requests, issue and return resources, and record maintenance.
5. Demonstrate encapsulation, inheritance, abstraction, interfaces, polymorphism, exceptions, collections, file I/O, packages, multithreading, and JDBC.
6. Maintain persistent records using a lightweight relational database.
7. Generate simple reports and backups useful to a laboratory administrator.
8. Keep the application understandable, modular, and appropriate for a second-year Java course.

4. Functional Requirements

4.1 Resource and Experiment Management

- Add, update, search, and display laboratory resources.
- Maintain resource type, location, quantity, and status.
- Create and maintain experiment information.
- Associate required resources with experiments.
- Mark resources as available, reserved, issued, or under maintenance.

4.2 Booking and Availability Management

- Allow students to view available experiments and slots.
- Accept valid booking requests.
- Check student eligibility and booking validity.
- Detect conflicting bookings before creating a request.
- Approve or reject requests through the staff workflow.
- Support cancellation and waiting-list behavior where applicable.

4.3 Issue, Return and Maintenance

- Issue resources against approved bookings.
- Record resource returns.
- Record maintenance problems and status.
- Prevent the use of resources that are under maintenance or otherwise unavailable.

4.4 Reporting and Data Management

- Display booking history and resource usage information.
- Generate basic resource, booking, and maintenance summaries.
- Export selected reports to files.
- Perform a basic background backup operation.

5. Non-functional Requirements

Requirement	Description
Usability	The command-line menus use clear prompts and readable messages.
Reliability	Expected invalid inputs and application errors are handled without abrupt termination.
Maintainability	The project is split into packages by responsibility.
Performance	Normal college-sized operations should complete quickly.
Data Integrity	Booking and resource status changes are validated before persistence.
Error Handling	Custom and standard exceptions provide meaningful failure messages.
Resource Efficiency	Collections and file operations are used only where appropriate.
Extensibility	New resource types or business rules can be added without rewriting the entire system.

6. System Architecture

CampusLab follows a simple layered structure. The console interface accepts user input, the service layer applies business rules, the repository layer performs JDBC operations, and the database stores persistent records. File management and a small background task provide supporting I/O and multithreading functionality.

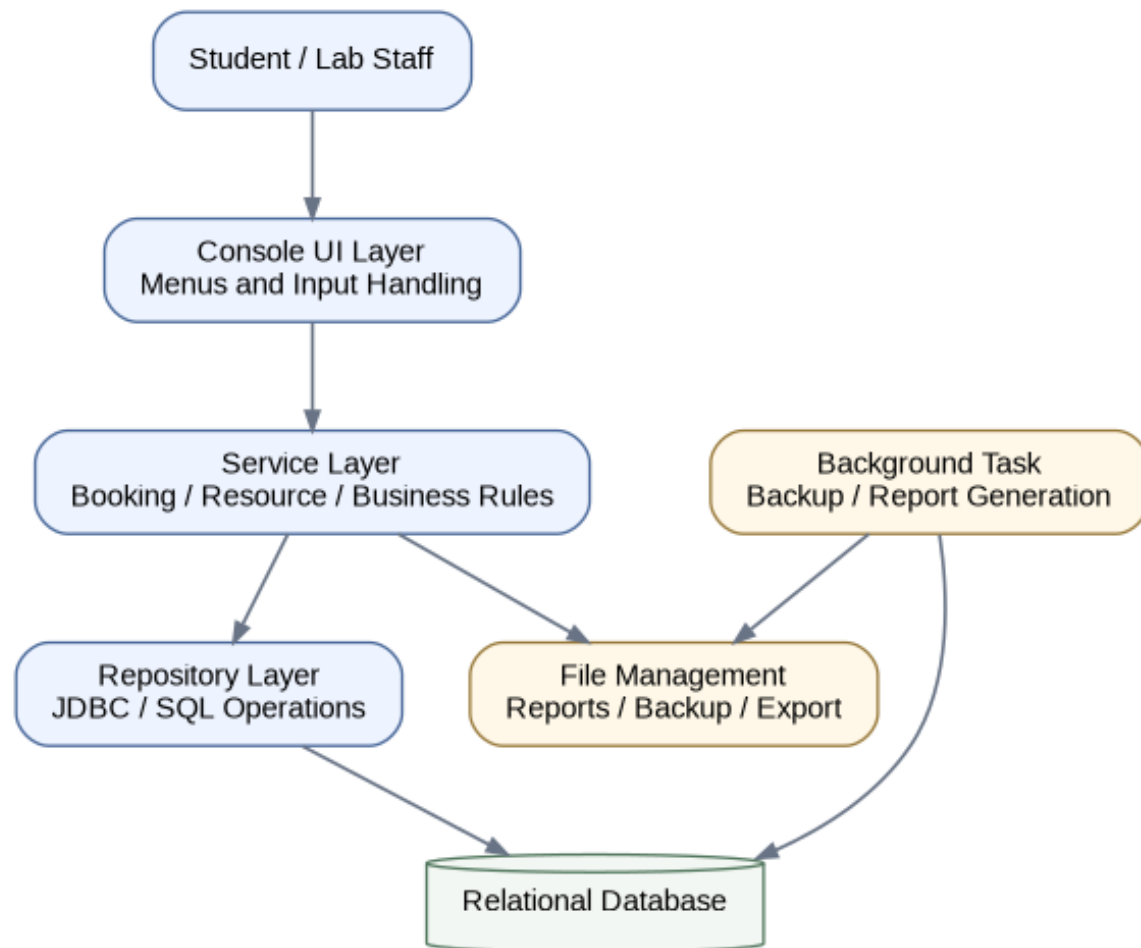


Figure 1. High-level system architecture.

7. All Design Diagrams

7.1 Workflow Diagram



Figure 2. Main booking workflow.

7.2 Use Case Diagram

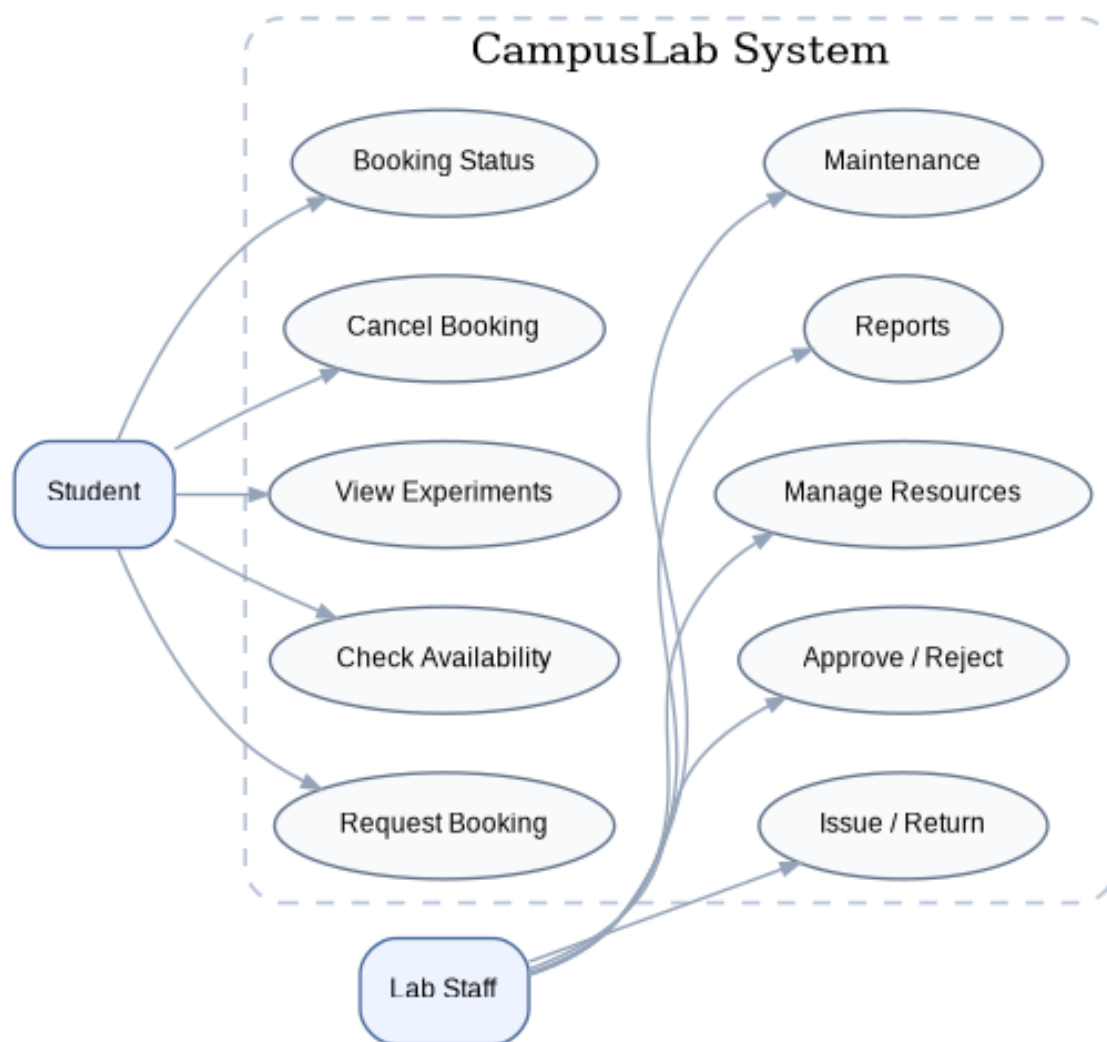


Figure 3. Use cases for Student and Lab Staff actors.

7.3 Class Diagram

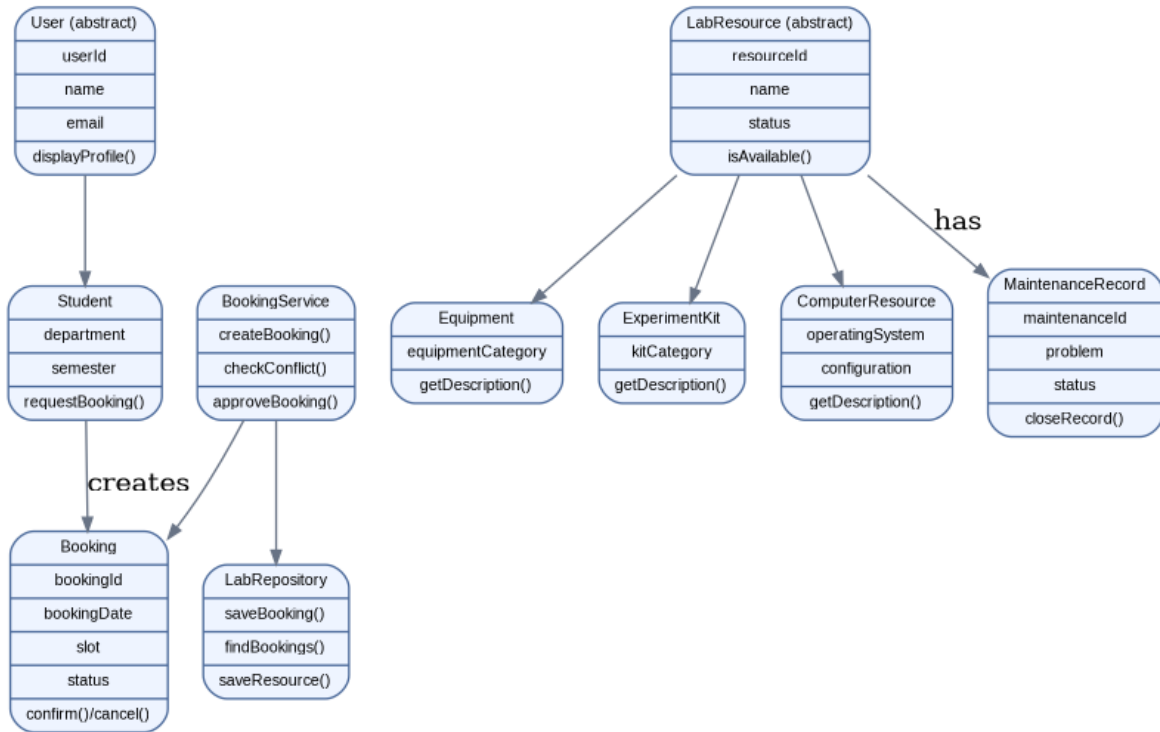


Figure 4. Core object-oriented class relationships.

7.4 Sequence Diagram

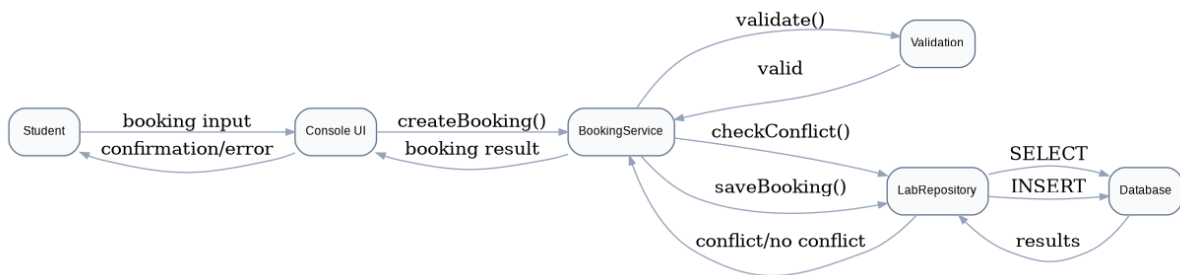


Figure 5. Sequence for a booking request.

7.5 Component Diagram

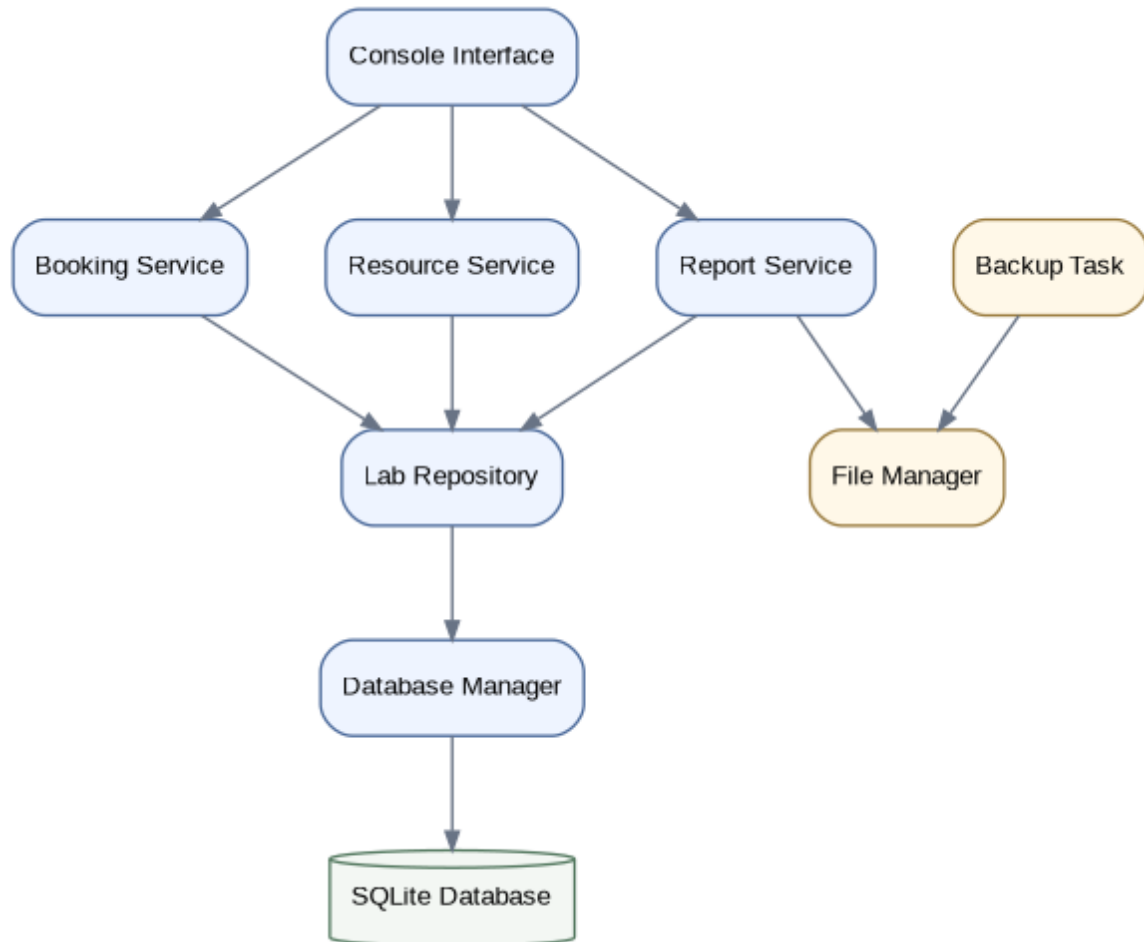


Figure 6. Software component relationships.

7.6 ER Diagram

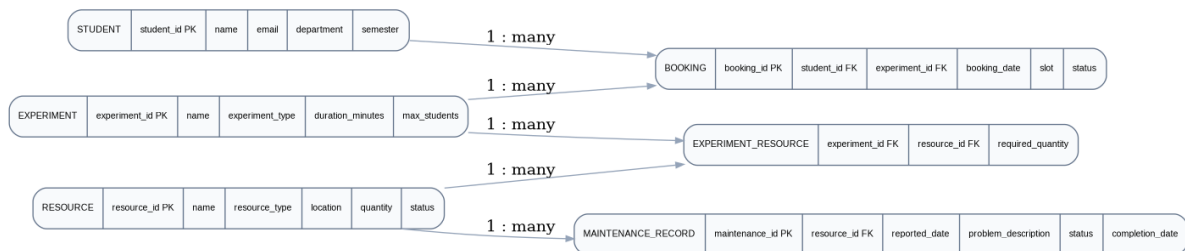


Figure 7. Relational data model.

8. Design Decisions

8.1 Core Java and CLI

A command-line interface was selected so that the project remains focused on Java language features and course concepts. No web framework or frontend technology is required.

8.2 JDBC and SQLite

JDBC is used because database connectivity and CRUD operations are explicitly included in the syllabus. SQLite keeps the project lightweight and easy to run on a student laptop while still providing relational persistence.

8.3 Layered organization

Database code is kept separate from business rules and the console menu. This makes the code easier to understand, test, and maintain.

8.4 Object-oriented domain model

The resource hierarchy is modeled using an abstract base class and concrete resource types. Interfaces represent capabilities, while services perform business operations.

8.5 File I/O

File operations are limited to useful tasks such as report export and backup. This demonstrates the I/O concepts without creating unnecessary file formats.

8.6 Multithreading

A small background task demonstrates thread creation and controlled concurrent work through backup/report processing. Multithreading is not used everywhere because that would add unnecessary complexity.

9. Implementation Details

9.1 Package Structure

```
src/main/java/com/campuslab/  
├── app/  
├── db/  
├── enums/  
├── exception/  
├── interfaces/  
├── model/  
├── repository/  
├── service/  
└── util/
```

9.2 Core Classes

Class / Area	Purpose
CampusLabApplication	Application entry point.
ConsoleMenu	Command-line menus and user interaction.
User / Student / LabStaff	User modeling and inheritance.
LabResource / Equipment / ExperimentKit / ComputerResource	Resource hierarchy and polymorphism.
Experiment	Experiment definition and required resources.
Booking	Booking information and status.
LabRepository	JDBC persistence operations.
BookingService	Booking validation and business logic.
ResourceService	Resource management logic.
DatabaseManager	Database connection and initialization.
FileManager	Report export and file handling.
BackupTask	Background task for backup/report work.

9.3 Java Concepts Demonstrated

- Encapsulation through private fields and methods.
- Inheritance through User and LabResource hierarchies.
- Abstraction using abstract classes.
- Interfaces for resource capabilities.
- Polymorphism through parent references and overridden methods.
- Enums for resource and booking states.
- Collections for dynamic records.
- Custom exception classes for domain errors.
- Packages for modular organization.
- File streams for text export and backup.
- Multithreading through a Runnable background task.
- JDBC for database CRUD operations.

10. Testing Approach

Testing for CampusLab is centered on the main functional requirements of the semester project. The prepared test cases cover application startup, resource viewing, valid booking, invalid input, booking conflicts, cancellation, maintenance restrictions, return handling, and invalid menu selections.

Test ID	Scenario	Expected Result
TC-01	Start application	Main menu appears and initialization completes.
TC-02	View resources	Resource records are displayed correctly.
TC-03	Valid booking	Booking is created for valid input.
TC-04	Invalid student ID	Request is rejected with a clear message.
TC-05	Booking conflict	Conflicting request is blocked or handled by waitlist rules.
TC-06	Invalid date	Validation message is displayed and input can be retried.
TC-07	Cancel booking	Eligible booking changes to cancelled.
TC-08	Maintenance resource	Booking is prevented for unavailable resource.
TC-09	Resource return	Resource lifecycle is updated correctly.
TC-10	Invalid menu choice	Error is shown and menu remains usable.

Execution note: the project report should be updated with the actual Pass/Fail results after the final local run of the application. The test matrix is included so that the final verification can be completed quickly and consistently.

11. Challenges Faced

- Choosing a project scope that was useful enough to demonstrate many Java concepts without becoming an enterprise-scale application.
- Separating console interaction, business logic, and JDBC persistence into understandable packages.
- Designing booking conflict validation so that the application handles invalid requests without corrupting booking data.
- Using inheritance and interfaces in a way that represents the laboratory domain rather than adding artificial examples.
- Keeping file I/O and multithreading practical and limited to supporting tasks.

12. Learnings & Key Takeaways

- A real problem is easier to model when classes represent meaningful domain objects.
- Encapsulation and separation of responsibilities make the code easier to understand and modify.
- Interfaces and abstract classes can be useful when multiple objects share capabilities or common behavior.
- Exception handling is most valuable when it is connected to real validation and business rules.
- JDBC makes the relationship between Java objects and persistent data more concrete and understandable.
- Collections are useful for handling variable-size application data.
- Good project documentation is part of software engineering, not just a reporting requirement.

13. Future Enhancements

- Add a graphical user interface after GUI concepts are covered in the course.
- Add role-based login with stronger authentication.
- Provide email or notification support for booking approvals and reminders.
- Introduce richer analytics for resource utilization and maintenance trends.
- Move from a local SQLite database to a shared database server for multi-user deployment.
- Add automated unit and integration tests when a larger testing framework is introduced.

14. References

- Oracle. Java Documentation. <https://docs.oracle.com/en/java/>
- Oracle. JDBC Basics. <https://docs.oracle.com/javase/tutorial/jdbc/>
- Oracle. Java Object-Oriented Programming Concepts. <https://docs.oracle.com/javase/tutorial/java/concepts/>
- Oracle. Java I/O Streams. <https://docs.oracle.com/javase/tutorial/essential/io/>
- Oracle. Concurrency in Java. <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- SQLite Documentation. <https://www.sqlite.org/docs.html>
- Apache Maven Documentation. <https://maven.apache.org/guides/>